TP3 – Navigation et données distantes

React Native · En trinôme · Projet, jalon 2 · **Noté**

Votre projet a son architecture. Il devient maintenant une vraie application : plusieurs écrans reliés entre eux, des adresses sur le web, des données qui viennent d'une **API publique** et des écrans qui réagissent à leur cycle de vie.

Ce TP en bref

- **Objectifs** : mettre en place la navigation avec Expo Router (onglets, pile, route dynamique), consommer une API avec `fetch`, gérer le cycle de vie des écrans et des listes plus riches.
- En trinôme, dans le dépôt du projet, **en autonomie** du 29 octobre au 24 novembre.
- **Ce jalon est noté** sur 20 avec la grille de la dernière partie. Sa note compte pour **3 points** sur les 20 du projet.
- Le cours 3 (catalogue de séjours avec météo) vous sert de modèle.

Qui fait quoi ?

- **A · Catalogue** : étape 2 (API publique) et étape 4 (catalogue et filtres).
- **B · Navigation et compte** : étape 1 (navigation) et étape 3 (détail).
- **C · Données partagées** : la `SectionList` de l'étape 4 et l'étape 5 (cycle de vie).

1 La navigation

Expo Router · Stack · Tabs

1. Réorganisez `src/app/` :
   - une **pile racine** dans `src/app/_layout.tsx`, qui garde votre Provider ;
   - un groupe `(tabs)` avec **au moins deux onglets**, chacun avec un titre et une icône ;
   - une **route dynamique** pour le détail d'un élément, par exemple `src/app/recipes/[id].tsx` ;
   - un écran `+not-found.tsx` avec un lien de retour à l'accueil.
2. Déclarez `unstable_settings = { anchor: "(tabs)" }` dans la pile racine, pour qu'un détail ouvert par son URL garde une flèche de retour.

> Importez `Tabs` depuis `expo-router/js-tabs`. Installez les icônes avec `npx expo install @expo/vector-icons`, jamais avec `npm install` : la commande d'Expo choisit la version compatible avec votre SDK.

2 L'API publique

fetch · async · typage

1. Choisissez l'API de votre sujet et notez son adresse dans le README. Elle doit être **gratuite et sans clé secrète**.
2. Ouvrez une réponse de l'API dans votre navigateur. Écrivez le **type de la réponse**, limité aux champs que vous utilisez.
3. Remplacez le contenu de votre service par un appel à `fetch` :
   - vérifiez `response.ok` et levez une erreur sinon ;
   - **convertissez** la réponse en objets de votre modèle. Le reste de l'application ne doit pas connaître les noms de champs de l'API.
4. Vérifiez que vos hooks et vos composants **n'ont pas changé**. Si vous avez dû les modifier, c'est qu'une responsabilité était mal placée au TP2.
5. Coupez le réseau (mode avion ou outils de développement du navigateur) : l'erreur et le bouton « Réessayer » doivent s'afficher.

Indice : l'API renvoie beaucoup plus de champs que nécessaire

C'est normal. Ne typez que ce que vous utilisez, puis construisez vos objets du modèle avec les bons noms. Un champ facultatif de l'API devient une propriété facultative (`?`) de votre type.

Indice : les données locales servent encore

Si votre API ne fournit pas tout (vos catégories, par exemple), gardez ces données dans `src/data/` . Ce sont vos deux CRUD Firebase du TP4 qui remplaceront le reste.

3 L'écran de détail

useLocalSearchParams · liens profonds

1. Depuis le catalogue, un appui sur un élément ouvre son détail avec `router.push` ou un `<Link>`.
2. Le détail lit l'identifiant avec `useLocalSearchParams` et **recharge l'élément** par un service et un hook dédiés.
3. Son titre apparaît dans l'en-tête (`<Stack.Screen options>`).
4. Un identifiant inconnu affiche un message clair, sans planter.

> **Le test qui compte**
>
> Sur le web, ouvrez un détail, puis **rechargez la page** (F5) et copiez son adresse dans un nouvel onglet. Le même détail doit s'afficher, avec une flèche de retour.

4 Le catalogue et ses filtres

FlatList · SectionList · hooks

1. Le catalogue affiche les données de l'API en grille sur ordinateur et en liste sur téléphone.
2. Il propose **au moins deux filtres combinables** (une recherche et une catégorie, par exemple). Leur logique est dans le modèle et dans un hook, jamais dans un composant.
3. Un des écrans utilise une `SectionList` pour présenter des éléments **groupés** (par catégorie, par date…). La fonction de regroupement est dans le modèle.
4. Chaque liste a une `keyExtractor`, un message quand elle est vide, et aucun calcul dans `renderItem`.

5 Le cycle de vie

useFocusEffect · AppState

1. Repérez un écran dont les données peuvent changer pendant qu'il est caché (un onglet de favoris, par exemple). Rechargez-le avec `useFocusEffect`.
2. Dans le README, expliquez en deux ou trois phrases **pourquoi** un `useEffect(…, [])` ne suffisait pas à cet endroit.
3. **Facultatif** : rafraîchissez une donnée au retour de l'application au premier plan, avec `AppState`.

✓ Rendu et point d'étape

- ☐ `npx tsc --noEmit` et `npx expo lint` n'affichent aucune erreur.
- ☐ L'application se lance sur le web et sur Expo Go.
- ☐ Le README liste les écrans et leurs URL, l'API utilisée et l'explication de l'étape 5.
- ☐ Chaque membre a commité sur sa partie pendant ce jalon.

1. Quand le jalon est prêt, créez un **tag Git** et poussez-le, **au plus tard le mardi 24 novembre à 23h59** :

   Terminal

   ```
   git tag jalon-2
   git push origin jalon-2
   ```

2. Envoyez un e-mail à chevalier@chris-freelance.com avec l'objet `[RN-IWB1] Jalon 2 — NOM1, NOM2, NOM3` (ou `IWB2`).
3. Le mercredi 25 novembre, pendant le TP4, je passe voir chaque groupe pour le **point d'étape** : 5 minutes de démonstration où chacun montre sa partie, puis une question à chaque membre sur son code.

Je corrige le commit pointé par le tag `jalon-2`. Les commits suivants comptent pour le rendu final, pas pour ce jalon.

20 La grille de notation

Critère	Ce que je vérifie	Points
Navigation	Pile racine, au moins deux onglets avec icônes, route dynamique de détail, écran `+not-found`	5
URL et liens profonds	Un détail s'ouvre et se recharge depuis son adresse, avec une flèche de retour	2
API publique	Service avec `fetch`, test de `response.ok`, réponse typée et convertie en modèle, états de chargement et d'erreur	5
Catalogue et listes	Grille ou liste selon la largeur, deux filtres combinables dans un hook, une `SectionList`	3
Cycle de vie	`useFocusEffect` utilisé à bon escient et expliqué dans le README	2
Architecture et qualité	MVC respecté (aucun service dans un composant), aucune erreur TypeScript, chaque membre a commité sur sa partie	3
Total		**20**

Un critère est noté en entier s'il fonctionne **sur le web et sur mobile**. Une fonctionnalité que personne ne sait expliquer au point d'étape n'est pas comptée.

La note du jalon est celle du groupe, **modulée pour chaque membre** selon l'état de sa partie et sa réponse au point d'étape.